call it feedData. This will be an XMLList-Collection object which can directly be used by our list later on.

```
[Bindable]
private var
feedData:XMLListCollection;
```

We are adding a metadata tag "Bindable" to this variable; this will tell Flex to generate code that will automatically update any components bound to this variable. This way whenever this feedData variable is changed so will the list displaying the data, with no effort on our part!

Now for the `onFeedLoaded` function that will be called when the feed is done loading:

```
private function
onFeedLoaded(event:Event):void
{
var rssFeed:XML = XML((event.
target as URLLoader).data);
feedData = new
XMLListCollection(rssFeed.
channel.item);
}
```

In this function:

1. `var rssFeed:XML = XML((event.target as URLLoader).data);` The `event.target` element is the URLLoader object with which we loaded the feed. We are casting this as a URLLoader and accessing its data property, which has the raw feed data. This data is being cast as an XML object and stored in the rssFeed XML variable.
2. `feedData = new XMLListCollection(rssFeed.channel.item);` Our bindable feedData variable is now being assigned a list of all the items in the RSS feed. The structure of the RSS has a root RSS element that includes one channel element with all our articles as item elements within. `rssFeed.channel.item` is thus a list of all the items in the RSS feed, and this is being wrapped into `XMLListCollection`.

We need to add a change handler to the List that will update the HTML component with article extract from the selected item in the list. The code for that is as follows:

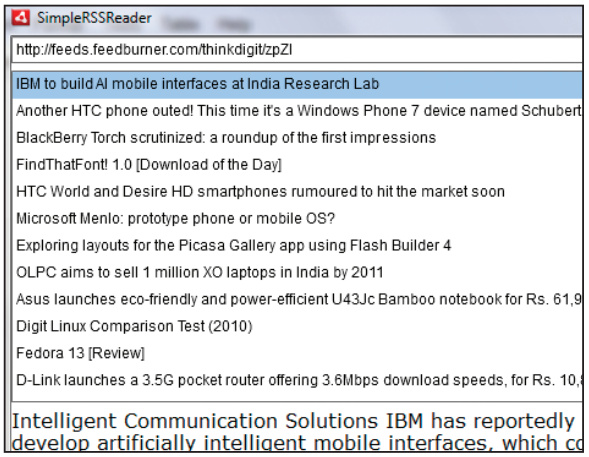
```
protected function
feedItems_changeHandler(
event:IndexChangeEvent):void
{
article.htmlText = feedItems.
selectedItem.description;
}
```

That's it! All we are doing is telling the WebKit engine added to the application using the HTML tag to use the content from the description property of the feed element selected in the `feedItems` list.

To tie everything in place, we need to add `labelField="title"` and `dataProvider="{feedData}"` to our list. The `labelField` attribute specifies that the "`title`" tag in the item element is what is to be displayed in the list. The dataProvider attribute binds the feedData variable as the data source of the list.

Here is what your final code should look like:

```
<?xml version="1.0"
encoding="utf-8"?>
<s:WindowedApplication
xmlns:fx="http://ns.adobe.com/
mxml/2009"
xmlns:s="library://ns.adobe.
com/flex/spark"
xmlns:mx="library://ns.adobe.
com/flex/mx"
width="800" height="600"
>
<s:layout>
<s:VerticalLayout />
</s:layout>
<fx:Script>
<![CDATA[
import mx.collections.
XMLListCollection;
import spark.events.
IndexChangeEvent;
[Bindable]
private var
feedData:XMLListCollection;
protected function loadFeed_
clickHandler(event:MouseEvent):
void {
var ul:URLLoader = new
URLLoader();
ul.addEventListener(Event.
COMPLETE, onFeedLoaded);
ul.load(new
URLRequest(feedUrl.text));
}
private function
onFeedLoaded(event:Event):void {
var rssFeed:XML = XML((event.
target as URLLoader).data);
feedData = new
XMLListCollection(rssFeed.
channel.item);
}
protected function feedItems_
changeHandler(event:IndexChangeE
vent):void {
article.htmlText = feedItems.
selectedItem.description;
}
]]>
</fx:Script>
<s:HGroup width="100%">
<s:TextInput id="feedUrl"
width="100%" />
<s:Button id="loadFeed"
label="Load" click="loadFeed_
clickHandler(event)" />
</s:HGroup>
<s:List id="feedItems"
dataProvider="{feedData}"
labelField="title"
width="100%" height="100%"
change="feedItems_
changeHandler(event)" />
<mx:HTML id="article"
width="100%" height="100%" />
</s:WindowedApplication>
```



The UI for our RSS reader application.

After this if you run the application, you will see a fully functional RSS reader which will display content from any RSS compliant feed.

Check out the extended version of this article online at http://www.think-digit.com/d/fb4_tut1_4 that adds two more features to this application.